Univerzitet u Beogradu

Elektotehnički fakultet

PROGRAMSKI PREVODIOCI 1

DOMAĆI ZADATAK

KOMPAJLER ZA MIKROJAVU – NIVO B

Marija Stakić 2021/0320

Opis postavke zadatka

Cilj projektnog zadatka je realizacija kompajlera za programski jezik MikroJava. Kompajler omogućava prevođenje sintaksno i semantički ispravnih MikroJava programa u izvršni bajtkod namenjen izvršavanju na MikroJava virtuelnoj mašini (MJVM).

Kompajler je realizovan kroz četiri osnovne faze:

1) Leksička analiza – izdvajanje tokena iz izvornog programa,
2) Sintaksna analiza – proveravanje gramatičke ispravnosti programa i formiranje apstraktnog sintaksnog stabla (AST),
3) Semantička analiza – provera kontekstnih uslova i ažuriranje tabele simbola,
4) Generisanje koda – generisanje izvršnog .obj fajla za MJVM.

U okviru projekta implementirane su sve funkcionalnosti predviđene za Nivo B, uključujući rad sa promenljivama i nizovima, aritmetičke i logičke izraze, kontrolne strukture (if, for, switch), break i continue naredbe, kao i pozive globalnih funkcija. Implementacija je realizovana korišćenjem alata JFlex i AST-CUP, kao i biblioteka symboltable-1-1.jar i mj-runtime-1.1.jar.

Komande za generisanje, prevođenje, pokretanje i testiranje rešenja

Projekat koristi Apache Ant za automatizaciju procesa generisanja i prevođenja. Sve faze izgradnje definisane su u fajlu build.xml.
Proces izgradnje obuhvata sledeće korake:

- Generisanje leksičkog analizatora pomoću alata JFlex na osnovu specifikacije mjlexer.lex.
- Generisanje sintaksnog analizatora i klasa apstraktnog sintaksnog stabla pomoću AST-CUP alata na osnovu specifikacije mjparser.cup.
- Automatsko prilagođavanje paketa generisanih klasa.
- Prevođenje svih Java fajlova uz uključivanje potrebnih biblioteka (JFlex, AST-CUP, symboltable, mj-runtime).

Nakon uspešnog prevođenja moguće je pokrenuti kompajler nad proizvoljnim MikroJava programom, pri čemu se kao rezultat dobija izvršni .obj fajl za MikroJava virtuelnu mašinu.
Generisani objekat fajl može se:

- disasemblirati radi pregleda bajtkoda,
- izvršiti u debug režimu,
- pokrenuti u standardnom režimu rada.

Za potrebe testiranja korišćeni su ulazni fajlovi smešteni u direktorijumu test/. Standardni izlaz i izlaz greške mogu se preusmeriti u odgovarajuće .out i .err fajlove radi lakše analize rezultata i poređenja sa očekivanim ishodom.

Opis priloženih test primera

U okviru projekta korišćeni su test primeri koji proveravaju funkcionalnosti predviđene za Nivo A i Nivo B.
Test primer test301 obuhvata funkcionalnosti osnovnog nivoa i proverava:

- deklaracije globalnih i lokalnih promenljivih,
- deklaracije konstanti,
- rad sa nizovima prostih tipova,
- aritmetičke izraze,
- dodelu vrednosti,
- inkrement i dekrement operacije,
- read i print naredbe.

Test primer test302 obuhvata proširene funkcionalnosti Nivoa B i proverava:

- definisanje i pozive globalnih funkcija sa parametrima,
- povratne vrednosti funkcija,
- logičke i relacione izraze,
- if-else konstrukcije,
- for petlju,
- switch naredbu sa više case grana,
- break i continue naredbe,
- proveru tipova izraza u složenijim situacijama.

Standardno izvršavanje programa

Koristi se za proveru ispravnog rada generisanog koda:
java -cp lib/mj-runtime-1.1.jar rs.etf.pp1.mj.runtime.Run test/test301.obj < test/inputs.txt > test/test301.run.out 2> test/test301.run.err
java -cp lib/mj-runtime-1.1.jar rs.etf.pp1.mj.runtime.Run test/test302.obj < test/inputs.txt > test/test302.run.out 2> test/test302.run.err
java -cp lib/mj-runtime-1.1.jar rs.etf.pp1.mj.runtime.Run test/program.obj < test/inputs.txt > test/program.run.out 2> test/program.run.err
Ove komande:

- preuzimaju ulaz iz fajla inputs.txt,
- upisuju standardni izlaz u .run.out,
- upisuju greške u .run.err.

Pokretanje u debug režimu

Koristi se za detaljnu analizu izvršavanja bajtkoda:
java -cp lib/mj-runtime-1.1.jar rs.etf.pp1.mj.runtime.Run test/test301.obj -debug < test/inputs.txt > test/test301.rundbg.out 2> test/test301.rundbg.err
java -cp lib/mj-runtime-1.1.jar rs.etf.pp1.mj.runtime.Run test/test302.obj -debug < test/inputs.txt > test/test302.rundbg.out 2> test/test302.rundbg.err
java -cp lib/mj-runtime-1.1.jar rs.etf.pp1.mj.runtime.Run test/program.obj -debug < test/inputs.txt > test/program.rundbg.out 2> test/program.rundbg.err
Debug režim omogućava praćenje instrukcija i rada virtuelne mašine.

Disasembliranje objektnog fajla

Koristi se za prikaz generisanog bajtkoda:
java -cp lib/mj-runtime-1.1.jar rs.etf.pp1.mj.runtime.disasm test/program.obj > test/program.disasm.out 2> test/program.disasm.err
Ova komanda omogućava proveru da li je generator koda proizveo ispravne instrukcije.

Testovi za greške i oporavak

Pored izvršnih testova, pripremljeni su i:

- testovi sa semantičkim greškama (višestruke deklaracije, pogrešni tipovi, neispravni pozivi funkcija, nedeklarisani simboli),
- testovi za oporavak od sintaksnih grešaka (greške u deklaracijama, dodelama, formalnim parametrima i if uslovima).

Ovi testovi služe za proveru da kompajler pravilno detektuje greške, prijavljuje odgovarajuće poruke i nastavlja sa radom bez prekida.

Opis novouvedenih klasa

Compiler

Glavna klasa programa. Zadužena je za pokretanje svih faza prevođenja: leksičke, sintaksne i semantičke analize, kao i generisanja koda. Takođe omogućava ispis apstraktnog sintaksnog stabla i sadržaja tabele simbola.

SemanticAnalyzer

Nasleđuje klasu VisitorAdapter i realizuje semantičku analizu obilaskom apstraktnog sintaksnog stabla.

Obavlja:

- obradu deklaracija globalnih i lokalnih simbola,
- proveru višestrukih deklaracija,
- proveru tipova izraza,
- proveru kompatibilnosti pri dodeli vrednosti,
- proveru poziva funkcija,
- proveru kontrolnih struktura (if, for, switch),
- proveru break, continue i return naredbi.

Za rad sa simbolima koristi biblioteku symboltable-1-1.jar.

CodeGenerator

Nasleđuje VisitorAdapter i implementira generisanje MJVM bajtkoda.

Generiše instrukcije za:

- aritmetičke, relacione i logičke operacije,
- ternarni operator,
- rad sa nizovima,
- kontrolne strukture,
- pozive funkcija,
- ulazno-izlazne naredbe.

Rezultat generisanja je izvršni .obj fajl.

CounterVisitor

Klasa CounterVisitor implementira obilazak apstraktnog sintaksnog stabla radi prikupljanja statističkih informacija o programu.

Koristi se za brojanje:

- broja globalnih promenljivih,
- broja lokalnih promenljivih,
- broja metoda,
- broja poziva funkcija,
- broja iskaza u okviru programa.

Rezultati obilaska ispisuju se nakon uspešne sintaksne analize i služe kao dodatna provera ispravnosti parsiranja i formiranja stabla.

MJParser

Automatski generisana klasa pomoću AST-CUP alata. Implementira LALR(1) parser i formira apstraktno sintaksno stablo.

Yylex

Automatski generisana klasa pomoću JFlex alata. Implementira leksičku analizu i vraća tokene parseru.